

Aula 11 - 05/05/2026

Uso de Collections (List, Set, Map,
Queue/Deque)

Sumário

- Visão geral do *Java Collections Framework*
 - **List** (ArrayList, LinkedList) — ordenação, busca, remoção
 - **Set** (HashSet, LinkedHashSet, TreeSet) — unicidade, ordenação natural
 - **Map** (HashMap, LinkedHashMap, TreeMap) — chave/valor, iteração
 - **Queue/Deque** (ArrayDeque, PriorityQueue) — filas, pilhas, *undo/redo*
- Genéricos, **Comparator**, **Comparable**, **equals/hashCode**
- **Design Patterns em Ação**: Strategy (ordenação/filtragem), Factory Method (escolha de implementação), Command (histórico com *undo/redo*), Template Method (importadores)
- Exercícios práticos guiados
- Para Fixar: evoluindo o *Caixa Eletrônico OO* com Collections

Conceitos Fundamentais

- **Coleção** é um contêiner para objetos. O *framework* provê interfaces (List, Set, Map, Queue) e implementações (*ArrayList*, *HashMap*, etc.).
- **Classes e Complexidade:**
 - *ArrayList*: acesso indexado $O(1)$, inserções no meio $O(n)$.
 - *LinkedList*: inserções/remoções nas pontas $O(1)$, acesso aleatório $O(n)$.
 - *HashSet/HashMap*: operações médias $O(1)$; dependem de bons `hashCode/equals`.
 - *TreeSet/TreeMap*: ordenados por *comparador*; operações $O(\log n)$.
- **Genéricos:** `List<Produto>` evita *casts* e dá segurança de tipos.
- **Contrato `equals/hashCode`:** se dois objetos são iguais por `equals`, **devem** ter o mesmo `hashCode`. Essencial para *Set/Map*.
- **Iteração segura:** use `for (var x : lista)` ou `Iterator`. Modificar a coleção enquanto itera pode lançar `ConcurrentModificationException`.

Lists

Uma **List** é uma coleção **ordenada** que permite **elementos duplicados** e o **acesso posicional** (por índice).

Ela representa uma sequência de objetos, ideal para armazenar elementos em uma ordem específica, permitindo recuperar, inserir ou remover itens em posições determinadas.

Características principais:

- Mantém a **ordem de inserção**.
- Permite **valores repetidos**.
- Cada elemento tem uma **posição numérica (índice)**, começando em 0.
- Suas implementações mais comuns são:
 - **ArrayList**: baseada em vetor, mais eficiente para leitura e acesso aleatório.
 - **LinkedList**: baseada em nós encadeados, mais eficiente para inserções e remoções frequentes.
- Pode ser percorrida com **for-each**, **Iterator** ou **stream()**.

```
J Produto.java x
src > exemplo01 > J Produto.java > Produto > sku
1 package exemplo01;
2
3 public class Produto {
4     private final String sku;
5     private String nome;
6     private double preco;
7
8     public Produto(String sku, String nome, double preco) {
9         this.sku = sku;
10        this.nome = nome;
11        this.preco = preco;
12    }
13    public String getSku() { return sku; }
14    public String getNome() { return nome; }
15    public double getPreco() { return preco; }
16
17    @Override public String toString() {
18        return "%s - %s (R$ %.2f)".formatted(sku, nome, preco);
19    }
20 }

J App.java x
src > exemplo01 > J App.java > App
1 package exemplo01;
2
3 public class App {
4     Run | Debug
5     public static void main(String[] args) {
6         var c = new Catalogo();
7         c.add(new Produto(sku:"A1", nome:"Mouse", preco:79.9));
8         c.add(new Produto(sku:"B2", nome:"Teclado", preco:149.0));
9         c.add(new Produto(sku:"C3", nome:"Webcam", preco:299.0));
10
11        System.out.println(c.ordenarPor(Catalogo.POR_NOME));
12        System.out.println(c.ordenarPor(Catalogo.POR_PRECO_DESC));
13    }
14 }

J Catalogo.java x
src > exemplo01 > J Catalogo.java > Catalogo
1 package exemplo01;
2
3 import java.util.*;
4
5 public class Catalogo {
6     private final List<Produto> itens = new ArrayList<>();
7
8     public void add(Produto p) { itens.add(p); }
9
10    public List<Produto> ordenarPor(Comparator<Produto> estrategia) {
11        List<Produto> copia = new ArrayList<>(itens);
12        copia.sort(estrategia);
13        return copia;
14    }
15
16    // Estratégias
17    public static final Comparator<Produto> POR_NOME =
18        Comparator.comparing(Produto::getNome, String.CASE_INSENSITIVE_ORDER);
19
20    public static final Comparator<Produto> POR_PRECO_ASC =
21        Comparator.comparingDouble(Produto::getPreco);
22
23    public static final Comparator<Produto> POR_PRECO_DESC =
24        POR_PRECO_ASC.reversed();
25 }
```

List: ordenação e busca com Strategy (Comparator)

```
PS C:\Users\rllang\OneDrive\Documents\Java\JavaCollections> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rllang\OneDrive\Docu'
' exemplo01.App'
[A1 - Mouse (R$ 79,90), B2 - Teclado (R$ 149,00), C3 - Webcam (R$ 299,00)]
[C3 - Webcam (R$ 299,00), B2 - Teclado (R$ 149,00), A1 - Mouse (R$ 79,90)]
PS C:\Users\rllang\OneDrive\Documents\Java\JavaCollections> █
```

```

1 package exemplo02;
2
3 import java.util.*;
4
5 public class DemoSet {
6     public static void main(String[] args) {
7         // HashSet: rápido, sem ordem
8         Set<String> tags = new HashSet<>(List.of(e1:"java",e2:"oo",e3:"java",e4:"collections"));
9         System.out.println(tags); // sem duplicatas
10
11         // LinkedHashSet: preserva ordem de inserção
12         Set<String> ordemInsercao = new LinkedHashSet<>(List.of(e1:"b",e2:"a",e3:"c",e4:"a"));
13         System.out.println(ordemInsercao); // [b, a, c]
14
15         // TreeSet: ordenado (natural ou Comparator)
16         Set<String> ordenado = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
17         ordenado.addAll(List.of(e1:"Delta",e2:"alpha",e3:"Bravo"));
18         System.out.println(ordenado); // [alpha, Bravo, Delta]
19     }
20 }

```

Set: unicidade, ordenação natural e por comparador

✓ TERMINAL

```

PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaCollections> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe'
g\OneDrive\Documentos\Java\JavaCollections\bin' 'exemplo02.DemoSet'
• [oo, java, collections]
  [b, a, c]
  [alpha, Bravo, Delta]
○ PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaCollections>

```

Exercício 01: Catálogo de Produtos (List + Strategy)

Objetivo: organizar uma `List<Produto>` e trocar estratégias de ordenação/filtragem sem `if-else` gigante.

Enunciado:

1. Implemente `Catalogo` com `add`, `removerPorSku`, `buscarPorNome(parcial)` (retorna nova lista filtrada *case-insensitive*).
2. Crie pelo menos **3 estratégias** de ordenação (`Comparator`): por nome, por preço asc/desc, por SKU.
3. Na `App`, popular lista com 8 a 10 produtos e imprimir **3 visões** diferentes trocando a *strategy*.
4. **Desafio:** adicionar uma *strategy* de “quebra por faixa de preço” que devolve *sublistas* (≤ 100 , $100-300$, >300).

Dicas: `removeIf`, `toLowerCase`, `String.contains`, `Comparator.thenComparing`.


```
J Produto.java X
src > exercicio01 > J Produto.java > Produto
1 package exercicio01;
2
3 import java.util.*;
4
5 public class Produto {
6     private final String sku;
7     private String nome;
8     private double preco;
9
10    public Produto(String sku, String nome, double preco) {
11        this.sku = sku;
12        this.nome = nome;
13        this.preco = preco;
14    }
15    public String getSku() { return sku; }
16    public String getNome() { return nome; }
17    public double getPreco() { return preco; }
18
19    @Override
20    public String toString() {
21        return "%s - %s (R$ %.2f)".formatted(sku, nome, preco);
22    }
23 }
```

```
J App.java X
src > exercicio01 > J App.java > App
1 package exercicio01;
2
3 public class App {
4     Run | Debug
5     public static void main(String[] args) {
6         Catalogo c = new Catalogo();
7         c.adicionar(new Produto(sku:"A1", nome:"Mouse", preco:79.9));
8         c.adicionar(new Produto(sku:"B2", nome:"Teclado", preco:149.0));
9         c.adicionar(new Produto(sku:"C3", nome:"Webcam", preco:299.0));
10        c.adicionar(new Produto(sku:"D4", nome:"Monitor", preco:799.0));
11
12        System.out.println(x:"Por nome:");
13        c.ordenarPor(Catalogo.POR_NOME).forEach(System.out::println);
14
15        System.out.println(x:"\nPor preço desc:");
16        c.ordenarPor(Catalogo.POR_PRECO_DESC).forEach(System.out::println);
17
18        System.out.println(x:"\nBuscar 'mo':");
19        c.buscarPorNome(parcial:"mo").forEach(System.out::println);
20    }
21 }
```

```
J Catalogo.java X
src > exercicio01 > J Catalogo.java > Catalogo
1 package exercicio01;
2
3 import java.util.*;
4
5 public class Catalogo {
6     private final List<Produto> produtos = new ArrayList<>();
7
8     public void adicionar(Produto p) { produtos.add(p); }
9
10    public void removerPorSku(String sku) {
11        produtos.removeIf(p -> p.getSku().equalsIgnoreCase(sku));
12    }
13
14    public List<Produto> buscarPorNome(String parcial) {
15        return produtos.stream()
16            .filter(p -> p.getNome().toLowerCase().contains(parcial.toLowerCase()))
17            .toList();
18    }
19
20    public List<Produto> ordenarPor(Comparator<Produto> estrategia) {
21        return produtos.stream().sorted(estrategia).toList();
22    }
23
24    // Estratégias pré-definidas
25    public static final Comparator<Produto> POR_NOME =
26        Comparator.comparing(Produto::getNome, String.CASE_INSENSITIVE_ORDER);
27    public static final Comparator<Produto> POR_PRECO_ASC =
28        Comparator.comparingDouble(Produto::getPreco);
29    public static final Comparator<Produto> POR_PRECO_DESC = POR_PRECO_ASC.reversed();
30    public static final Comparator<Produto> POR_SKU =
31        Comparator.comparing(Produto::getSku);
32 }
```

OUTPUT DEBUG CONSOLE PORTS

TERMINAL

PS C:\Users\rnlang\OneDrive\Documents\Java\JavaCollections> &
'exercicio01.App'

● Por nome:
D4 - Monitor (R\$ 799,00)
A1 - Mouse (R\$ 79,90)
B2 - Teclado (R\$ 149,00)
C3 - Webcam (R\$ 299,00)

Por preço desc:
D4 - Monitor (R\$ 799,00)
C3 - Webcam (R\$ 299,00)
B2 - Teclado (R\$ 149,00)
A1 - Mouse (R\$ 79,90)

Buscar 'mo':
A1 - Mouse (R\$ 79,90)
D4 - Monitor (R\$ 799,00)

○ PS C:\Users\rnlang\OneDrive\Documents\Java\JavaCollections>

Maps

Um **Map** é uma estrutura de dados que **mapeia chaves a valores**, permitindo o acesso rápido a um valor a partir de uma chave única.

Diferente das listas, ele não armazena elementos de forma sequencial, mas sim em **pares (chave → valor)**.

Características principais:

- Cada **chave é única** dentro do mapa.
- Os **valores podem se repetir**.
- Permite **inserir, remover e buscar** por chave.
- Implementações mais comuns:
 - **HashMap**: rápido, sem ordem definida.
 - **LinkedHashMap**: mantém a ordem de inserção.
 - **TreeMap**: ordena as chaves de forma natural ou via **Comparator**.

J RepositorioCliente.java X

src > exemplo03 > J RepositorioCliente.java > PoliticaMap

```

1 package exemplo03;
2
3 import java.util.*;
4
5 public class RepositorioCliente {
6     private final Map<String, Cliente> porCpf;
7
8     public RepositorioCliente(Map<String, Cliente> impl) {
9         this.porCpf = impl;
10    }
11
12    public void salvar(Cliente c) { porCpf.put(c.getCpf(), c); }
13    public Cliente buscar(String cpf) { return porCpf.get(cpf); }
14    public Collection<Cliente> todos() { return porCpf.values(); }
15    public Cliente remover(String cpf) { return porCpf.remove(cpf); }
16
17    class Cliente {
18        private final String cpf; private final String nome;
19        public Cliente(String cpf, String nome){ this.cpf=cpf; this.nome=nome; }
20        public String getCpf(){ return cpf; } public String getNome(){ return nome; }
21    }
22
23    enum PoliticaMap { RAPIDO, ORDEM_INSERCAO, ORDENADO; }
24
25    class MapFactory {
26        public static <K,V> Map<K,V> criar(PoliticaMap p) {
27            return switch (p) {
28                case RAPIDO -> new HashMap<>();
29                case ORDEM_INSERCAO -> new LinkedHashMap<>();
30                case ORDENADO -> new TreeMap<>();
31            };
32        }
33    }

```

J App.java X

src > exemplo03 > J App.java > App

```

1 package exemplo03;
2
3 public class App {
4     public static void main(String[] args) {
5         var repo = new RepositorioCliente(MapFactory.criar(PoliticaMap.ORDEM_INSERCAO));
6         repo.salvar(new Cliente(cpf:"111", nome:"Ana"));
7         repo.salvar(new Cliente(cpf:"222", nome:"Bruno"));
8         for (var e : repo.todos()) System.out.println(e.getNome());
9     }
10 }

```

Map: chave/valor, iteração e lookups

OUTPUT DEBUG CONSOLE PORTS

TERMINAL

```

PS C:\Users\r\lrg\OneDrive\Documentos\Java\JavaCollections> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\r\lrg\OneDrive\Documentos\Java\JavaCollections\
Ana
Bruno
PS C:\Users\r\lrg\OneDrive\Documentos\Java\JavaCollections>
fwd-i-search: _

```

Exercício 02: Agenda/CRM (Map + Factory Method)

Objetivo: comparar *HashMap*, *LinkedHashMap* e *TreeMap* usando uma *factory*.

Enunciado:

1. Modele `Cliente { cpf, nome, email }` (CPF como chave).
2. `RepositorioCliente` deve ser criado com `MapFactory` (políticas: *RAPIDO*, *ORDEM_INSERTAO*, *ORDENADO*).
3. Implementar: `salvar`, `buscar`, `remover`, `listarTodosOrdenadosPorNome()` (dica: `new ArrayList<>(map.values()) + sort`).
4. **Desafio:** impedir cadastro de e-mails duplicados (use `Set<String>` auxiliar ou varredura por `values()`).

Padrão aplicado: **Factory Method** para desacoplar a escolha da implementação de `Map`.

```

J Cliente.java x
src > exercicio02 > J Cliente.java > Cliente
1 package exercicio02;
2
3 public class Cliente {
4     private final String cpf;
5     private final String nome;
6     private final String email;
7
8     public Cliente(String cpf, String nome, String email) {
9         this.cpf = cpf; this.nome = nome; this.email = email;
10    }
11    public String getCpf() { return cpf; }
12    public String getNome() { return nome; }
13    public String getEmail() { return email; }
14
15    @Override
16    public String toString() {
17        return "%s - %s <%s>".formatted(cpf, nome, email);
18    }
19 }

```

```

J PoliticaMap.java x
src > exercicio02 > J PoliticaMap.java > PoliticaMap > ORDENADO
1 package exercicio02;
2
3 public enum PoliticaMap {
4     RAPIDO,
5     ORDEM_INSERCAO,
6     ORDENADO;
7 }

```

```

J MapFactory.java x
src > exercicio02 > J MapFactory.java > MapFactory
1 package exercicio02;
2
3 import java.util.*;
4
5 public class MapFactory {
6     public static <K, V> Map<K, V> criar(PoliticaMap politica) {
7         return switch (politica) {
8             case RAPIDO -> new HashMap<>();
9             case ORDEM_INSERCAO -> new LinkedHashMap<>();
10            case ORDENADO -> new TreeMap<>();
11        };
12    }
13 }

```

```

J RepositorioCliente.java x
src > exercicio02 > J RepositorioCliente.java > RepositorioCliente
1 package exercicio02;
2
3 import java.util.*;
4
5 public class RepositorioCliente {
6     private final Map<String, Cliente> clientes;
7
8     public RepositorioCliente(PoliticaMap politica) {
9         this.clientes = MapFactory.criar(politica);
10    }
11
12    public void salvar(Cliente c) {
13        if (clientes.values().stream().anyMatch(x -> x.getEmail().equalsIgnoreCase(c.getEmail()))) {
14            throw new IllegalArgumentException("E-mail já cadastrado: " + c.getEmail());
15        }
16        clientes.put(c.getCpf(), c);
17    }
18
19    public Cliente buscar(String cpf) { return clientes.get(cpf); }
20    public Collection<Cliente> todos() { return clientes.values(); }
21    public void remover(String cpf) { clientes.remove(cpf); }
22
23    public List<Cliente> listarPorNome() {
24        return clientes.values().stream()
25            .sorted(Comparator.comparing(Cliente::getNome))
26            .toList();
27    }
28 }

```

```

J App.java x
src > exercicio02 > J App.java > App
1 package exercicio02;
2
3 public class App {
4     Run | Debug
5     public static void main(String[] args) {
6         RepositorioCliente repo = new RepositorioCliente(PoliticaMap.ORDEM_INSERCAO);
7         repo.salvar(new Cliente(cpf:"111", nome:"Ana", email:"ana@ex.com"));
8         repo.salvar(new Cliente(cpf:"222", nome:"Bruno", email:"bruno@ex.com"));
9         repo.salvar(new Cliente(cpf:"333", nome:"Carlos", email:"carlos@ex.com"));
10
11        System.out.println(x:"Todos:");
12        repo.todos().forEach(System.out::println);
13
14        System.out.println(x:"\nOrdenados por nome:");
15        repo.listarPorNome().forEach(System.out::println);
16    }

```

OUTPUT	DEBUG CONSOLE	PORTS
TERMINAL PS C:\Users\rnlang\OneDrive\Documents\Java\JavaCollections> 'exercicio02.App' • Todos: 111 - Ana <ana@ex.com> 222 - Bruno <bruno@ex.com> 333 - Carlos <carlos@ex.com> Ordenados por nome: 111 - Ana <ana@ex.com> 222 - Bruno <bruno@ex.com> 333 - Carlos <carlos@ex.com> © PS C:\Users\rnlang\OneDrive\Documents\Java\JavaCollections>		

Queue/Deque: filas, pilhas e undo/redo

Queue

- **O que é:** uma *fila* (primeiro a entrar, primeiro a sair — **FIFO**).
- **Para que serve:** processar itens em ordem de chegada (ex.: tarefas, BFS em grafos).
- **Java (interface `Queue<E>`):**
`offer(e)` insere; `poll()` remove e retorna o primeiro; `peek()` olha sem remover.
Implementações comuns: `ArrayDeque`, `LinkedList`, `PriorityQueue`* (*prioriza por chave, não é FIFO puro).

Deque

- **O que é:** *double-ended queue* (“fila de duas pontas”, pronuncia “**deck**”).
Insere/remove **no início e no fim** — pode agir como **fila (FIFO)** ou **pilha (LIFO)**.
- **Para que serve:** históricos *undo/redo*, janelas deslizantes, tanto pilha quanto fila com a mesma estrutura.
- **Java (interface `Deque<E>`):**
Início: `addFirst/offerFirst`, `removeFirst/pollFirst`, `getFirst/peekFirst`
Fim: `addLast/offerLast`, `removeLast/pollLast`, `getLast/peekLast`
Recomendada para **pilha** moderna: use `Deque/ArrayDeque` (evite `Stack` legado).

Queue/Deque: filas, pilhas e undo/redo

J Conta.java

src > exemplo04 > J Conta.java > Sacar

```
1 package exemplo04;
2
3 interface Comando {
4     void executar();
5     void desfazer();
6     String nome();
7 }
8 class Conta {
9     private double saldo;
10    public void depositar(double v){ saldo+=v; }
11    public void sacar(double v){ saldo-=v; }
12    public double getSaldo(){ return saldo; }
13 }
14 class Depositatar implements Comando {
15     private final Conta c; private final double v;
16     public Depositatar(Conta c,double v){ this.c=c; this.v=v; }
17     public void executar(){ c.depositar(v); }
18     public void desfazer(){ c.sacar(v); }
19     public String nome(){ return "Depositatar " + v; }
20 }
21 class Sacar implements Comando {
22     private final Conta c; private final double v;
23     public Sacar(Conta c,double v){ this.c=c; this.v=v; }
24     public void executar(){ c.sacar(v); }
25     public void desfazer(){ c.depositar(v); }
26     public String nome(){ return "Sacar " + v; }
27 }
```

J HistoricoTransacoes.java

src > exemplo04 > J HistoricoTransacoes.java > HistoricoTransacoes

```
1 package exemplo04;
2
3 import java.util.*;
4
5 public class HistoricoTransacoes {
6     private final Deque<Comando> feitos = new ArrayDeque<>();
7     private final Deque<Comando> desfeitos = new ArrayDeque<>();
8
9     public void executar(Comando c){
10         c.executar();
11         feitos.push(c);
12         desfeitos.clear();
13     }
14     public void undo(){
15         if(!feitos.isEmpty()){
16             var c = feitos.pop();
17             c.desfazer();
18             desfeitos.push(c);
19         }
20     }
21     public void redo(){
22         if(!desfeitos.isEmpty()){
23             var c = desfeitos.pop();
24             c.executar();
25             feitos.push(c);
26         }
27     }
28 }
```

Exercício 03: Transações com undo/redo

Objetivo: implementar histórico de operações da conta e permitir desfazer/refazer.

Enunciado:

1. Reaproveite `Conta`, `Depositar`, `Sacar` e `HistoricoTransacoes` (seção 5).
2. Na `App`, apresente um menu: **[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair**.
3. Cada ação gera um `Comando` e passa por `HistoricoTransacoes`.
4. **Desafio:** implemente `Transferir` (entre duas contas) como `Comando` composto (executa sacar+depositar; *desfazer* faz o inverso).

Padrão aplicado: **Command + Deque** (pilhas) para histórico.

Conta.java

```
src > exercicio03 > Conta.java > Conta
1 package exercicio03;
2
3 public class Conta {
4     private double saldo;
5
6     public void depositar(double v){ saldo += v; }
7     public void sacar(double v){ saldo -= v; }
8     public double getSaldo(){ return saldo; }
9
10    @Override
11    public String toString(){ return "Saldo: R$ %.2f".formatted(saldo); }
12 }
```

Depositar.java

```
src > exercicio03 > Depositar.java > Depositar
1 package exercicio03;
2
3 public class Depositar implements Comando {
4     private final Conta c;
5     private final double v;
6     public Depositar(Conta c, double v){ this.c=c; this.v=v; }
7
8     @Override public void executar(){ c.depositar(v); }
9     @Override public void desfazer(){ c.sacar(v); }
10    @Override public String nome(){ return "Depositar " + v; }
11 }
```

Sacar.java

```
src > exercicio03 > Sacar.java > Sacar
1 package exercicio03;
2
3 public class Sacar implements Comando {
4     private final Conta c;
5     private final double v;
6     public Sacar(Conta c, double v){ this.c=c; this.v=v; }
7
8     @Override public void executar(){ c.sacar(v); }
9     @Override public void desfazer(){ c.depositar(v); }
10    @Override public String nome(){ return "Sacar " + v; }
11 }
```

Comando.java

```
src > exercicio03 > Comando.java > Comando
1 package exercicio03;
2
3 public interface Comando {
4     void executar();
5     void desfazer();
6     String nome();
7 }
```

Historico.java

```
src > exercicio03 > Historico.java > Historico
1 package exercicio03;
2 import java.util.*;
3
4 public class Historico {
5     private final Deque<Comando> feitos = new ArrayDeque<>();
6     private final Deque<Comando> desfeitos = new ArrayDeque<>();
7
8     public void executar(Comando c) {
9         c.executar();
10        feitos.push(c);
11        desfeitos.clear();
12    }
13
14    public void undo() {
15        if (!feitos.isEmpty()) {
16            Comando c = feitos.pop();
17            c.desfazer();
18            desfeitos.push(c);
19        }
20    }
21
22    public void redo() {
23        if (!desfeitos.isEmpty()) {
24            Comando c = desfeitos.pop();
25            c.executar();
26            feitos.push(c);
27        }
28    }
29 }
```

```
J App.java 1 x
src > exercicio03 > J App.java > {} exercicio03
1 package exercicio03;
2 import java.util.Scanner;
3
4 public class App {
    Run | Debug
5     public static void main(String[] args) {
6         Conta conta = new Conta();
7         Historico hist = new Historico();
8         Scanner sc = new Scanner(System.in);
9
10        int op;
11        do {
12            System.out.println(x:"\n[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair");
13            op = sc.nextInt();
14
15            switch (op) {
16                case 1 -> { System.out.print(s:"Valor: "); double v=sc.nextDouble(); hist.executar(new Depositar(conta,v)); }
17                case 2 -> { System.out.print(s:"Valor: "); double v=sc.nextDouble(); hist.executar(new Sacar(conta,v)); }
18                case 3 -> System.out.println(conta);
19                case 4 -> hist.undo();
20                case 5 -> hist.redo();
21            }
22        } while (op != 0);
23    }
24 }
```

OUTPUT DEBUG CONSOLE PORTS

TERMINAL

```
PS C:\Users\rlang\OneDrive\Documents\Java\JavaCollections> & 'C:\Program Files\OpenJDK\jdk-22.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionior' 'exercicio03.App'
```

```
[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair
1
Valor: 100
```

```
[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair
3
Saldo: R$ 100,00
```

```
[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair
2
Valor: 20
```

```
[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair
3
Saldo: R$ 80,00
```

```
[1] Depositar [2] Sacar [3] Saldo [4] Undo [5] Redo [0] Sair
```

Para Fixar: Caixa Eletrônico com Collections

Objetivo: consolidar aulas 04–10 (encapsulamento, herança/interfaces, exceções, collections e padrões).

Descrição:

1. Mantenha **Conta** abstrata (**ContaCorrente**, **ContaPoupanca**) com exceções da Aula 09.
2. Gerencie **múltiplas contas** em memória com **Map<String, Conta>** (chave: número/CPF).
3. Use **List** para registrar *extratos* e **Deque** para *undo/redo* das transações (*Command*).
4. Ofereça diferentes **estratégias de extrato** (*Strategy*): por data, por valor, apenas débitos, apenas créditos.
5. Exponha uma *factory* para escolher **Map** conforme a política (rápido/ordenado).
6. **Bônus:** criar um pequeno **importador (Template Method)** que popula contas a partir de um CSV simplificado (número;tipo;saldo).

Critérios de avaliação sugeridos (rubrica):

- Correção funcional (40%)
- Aplicação dos padrões (30%)
- Qualidade de código e organização (20%)
- Experiência de uso (menu, mensagens, tratamento de erros) (10%)

